

# LSCBO Scheduler: Task-Reassignment Local Search for CBO-Based Multi-Cost Cloud Task Scheduling

First Author<sup>1\*</sup>

<sup>1\*</sup>Department or School, University Name, City, China.

Corresponding author(s). E-mail(s): [first.author@example.com](mailto:first.author@example.com);

## Abstract

Effective task scheduling is essential for reducing resource waste, operating cost, and service delay in cloud computing environments. The complexity of cloud task scheduling requires algorithms that can search globally while also exploiting the structure of task-to-VM assignment. This paper proposes LSCBO (Lévy-assisted Local-Search Coyote-Badger Optimization), a cloud-oriented scheduler that augments the Coyote-Badger Optimization (CBO) framework by combining task-reassignment local search with Lévy-assisted stochastic exploration. The task-reassignment operator redistributes tasks from overloaded to underloaded virtual machines before execution, while the Lévy perturbation supplies auxiliary global-search diversity.

The proposed scheduler is evaluated in two experimental series following the standard validation pattern for cloud meta-heuristic scheduling studies. First, LSCBO is tested on 29 CEC2017 benchmark functions to verify its general optimization capability. In this setting, LSCBO obtains the lowest Friedman average rank (1.86) within the seven-optimizer comparison, with statistically significant wins over CBO, PSO, SA, and GWO ( $p < 0.001$ ), and statistical parity with DE ( $p = 0.059$ ) and GTO ( $p = 0.481$ ). Second, LSCBO is applied to multi-cost cloud task scheduling in a CloudSim Plus scheduling pipeline—with all 12,780 formal rows executed through full CloudSim Plus simulation—and compared with eight population-based meta-heuristic schedulers under identical task, VM, seed, population-size, and iteration settings; the additional local-search evaluations used by LSCBO are explicitly counted and reported rather than absorbed into a nominally identical evaluation budget.

In the formal 12,780-row cloud scheduling experiment, LSCBO obtains the lowest average objective rank of 1.087 within the nine-scheduler meta-heuristic comparison. Relative to the standardized CBO baseline, LSCBO improves makespan by 9.6%–30.1%, energy by 7.3%–24.1%, load imbalance by 8.8%–24.7%, and the overall scalarized objective by 7.4%–22.6% across the five task scales. Component analysis shows that task-reassignment local search is the dominant performance

driver, while Lévy perturbation supplies a smaller and less stable auxiliary contribution. Further robustness tests across scalarization weights and VM heterogeneity settings show that LSCBO maintains the best average objective rank within the evaluated meta-heuristic scheduler family. These results support the narrower claim that combining a CBO search framework with scheduling-specific task reassignment is an effective design pattern within the evaluated family of cloud meta-heuristic schedulers.

**Keywords:** Cloud Task Scheduling, Coyote-Badger Optimizer, Lévy Flight, Task-Reassignment Local Search, CEC2017 Benchmarks, Meta-Heuristic Scheduling

# 1 Introduction

## 1.1 Background and Motivation

Cloud computing platforms must continuously assign thousands of heterogeneous tasks to heterogeneous Virtual Machines (VMs) while optimizing multiple objectives including makespan, energy consumption, monetary cost, and load balance. With the rapid growth of Internet of Things (IoT), big data analytics, and artificial intelligence workloads, the resulting Cloud Task Scheduling Problem (CTSP) has become increasingly large-scale and complex, rendering exact optimization methods such as branch-and-bound or dynamic programming impractical for real-time scheduling decisions [1, 3].

Population-based meta-heuristics have emerged as a practical alternative for solving the CTSP. These algorithms can provide high-quality approximate solutions across diverse problem domains without requiring gradient information or problem-specific exact formulations. Recent years have witnessed a surge of novel nature-inspired optimizers, including Harris Hawks Optimization (HHO) [10], Whale Optimization Algorithm (WOA) [11], Grey Wolf Optimizer (GWO) [35], Dung Beetle Optimizer (DBO) [12], Artificial Gorilla Troops Optimizer (GTO) [34], and Coyote-Badger Optimization (CBO) [24]. Many of these algorithms employ phase-based search structures that decompose the optimization process into distinct behavioral stages, and several scheduling-oriented variants have incorporated additional operators such as Lévy flight, chaotic mapping, or problem-specific heuristics to improve convergence [23, 25, 32, 33].

Despite these advances, three key challenges remain: (1) maintaining an effective balance between global exploration and local exploitation in high-dimensional scheduling landscapes, (2) ensuring rigorous and transparent statistical evaluation protocols for algorithm comparisons, and (3) providing systematic component-level analysis that identifies which algorithmic mechanisms genuinely contribute to performance.

## 1.2 Proposed Scheduler: LSCBO

To address these challenges, we propose LSCBO (Lévy-assisted Local-Search CBO), a population-based meta-heuristic scheduler that extends the CBO framework with two enhancements: (1) a greedy task-reassignment local search that exploits the scheduling problem structure by redistributing tasks from overloaded to underloaded VMs before

execution, and (2) Lévy-flight exploration that replaces the original deterministic tanh-based searching operator and provides stochastic perturbation for global search. The name encodes these two additions to CBO: the “L” denotes the Lévy-flight search and the “S” the task-reassignment local search. We emphasize at the outset that, although the Lévy component is retained in the algorithm name, the empirical analysis in this paper (perturbation-mechanism comparison in Section 4.2.7 and component ablation in Section 4.2.8) identifies task-reassignment local search as the dominant performance driver; Lévy flight is kept as an auxiliary search-diversity mechanism and is *not* claimed as an independent source of performance gain.

CBO is selected as the base framework for its modular three-phase architecture (searching, encircling, attacking), which enables controlled component modification without disrupting other phases. LSCBO preserves the encircling and attacking phases of CBO unchanged while enhancing the searching phase with stochastic exploration and augmenting the overall optimization process with problem-specific exploitation. The resulting scheduler follows the common cloud meta-heuristic design pattern: a general optimizer provides global search, while a scheduling-specific operator improves feasible task-to-VM assignments. The experiments below therefore interpret Lévy flight as an auxiliary search perturbation and task reassignment as the principal cloud-scheduling mechanism.

### 1.3 Contributions

The main contributions of this work are:

- **A cloud-oriented CBO scheduler:** We propose LSCBO, a CBO-based task scheduler that combines task-reassignment local search with Lévy-flight stochastic exploration for multi-cost cloud task scheduling.
- **Comprehensive two-series experimental validation:** We evaluate LSCBO through two complementary experimental campaigns: (i) 29 CEC2017 global optimization functions (F1, F3–F30) to validate its general optimization capability, and (ii) formal multi-cost scheduling experiments against population-based meta-heuristic schedulers across 5 task scales, 5 weight configurations, 3 VM heterogeneity regimes, perturbation mechanism comparisons, and component-level ablation.
- **Task-reassignment local search for cloud scheduling:** A greedy local-search operator that exploits the scheduling structure by reassigning tasks from the most-loaded to the least-loaded VM is shown to be the dominant performance driver in the cloud scheduling experiments; Lévy perturbation is treated as a secondary global-search aid.
- **Systematic component ablation and evolution-path analysis:** A five-stage evolution-path experiment traces the development from CBO to LSCBO, while component ablation separates the influence of local search, Lévy perturbation, and preserved CBO phases.
- **Robustness validation across optimization preferences and VM configurations:** LSCBO’s performance within the meta-heuristic scheduler family is empirically validated across five scalarization weight configurations and three VM heterogeneity regimes.

## 2 Related Work

### 2.1 Multi-Cost Cloud Task Scheduling Models

Cloud task scheduling has been studied through deterministic dispatching rules, population-based meta-heuristics, learning-based schedulers, and Pareto-front multi-objective methods. Classical rules such as Min-Min, Max-Min, and FCFS remain useful background methods because they are simple and fast [1, 3]. However, their fixed decision logic is often insufficient when heterogeneous VMs, large task sets, and several cost criteria must be considered together.

The central modeling issue is that a schedule should not be judged only by completion time. Energy consumption, monetary cost, and load imbalance also influence whether a task-to-VM assignment is useful in a cloud platform. Pareto-front approaches such as NSGA-II [39] and MOEA/D [40] directly approximate trade-off surfaces, whereas many cloud meta-heuristic schedulers use a normalized scalar objective so that several criteria can be optimized by one search process. The present paper follows the latter protocol and therefore evaluates LSCBO under a scalarized multi-cost model, while acknowledging that Pareto-front scheduling is a separate experimental setting.

### 2.2 Meta-Heuristic Schedulers for Cloud Task Assignment

Population-based meta-heuristics are widely used for cloud task scheduling because they can search large combinatorial spaces without requiring gradient information. Classical foundations include Particle Swarm Optimization (PSO) [4] and Genetic Algorithms (GA) [5]; in discrete scheduling, continuous positions are converted into task orders or assignment decisions through rules such as Smallest Position Value (SPV) [7]. More recent optimizers provide a diverse set of search templates, including Harris Hawks Optimization (HHO) [10], Whale Optimization Algorithm (WOA) [11], Grey Wolf Optimizer (GWO) [35], Dung Beetle Optimizer (DBO) [12], Kepler Optimization Algorithm (KOA) [14, 15], Arithmetic Optimization Algorithm (AOA) [16], Artificial Gorilla Troops Optimizer (GTO) [34], Snake Optimizer (SO) [17], Snow Ablation Optimizer (SAO) [13], Pelican Optimization Algorithm (POA) [18], Marine Predators Algorithm (MPA) [19], Reptile Search Algorithm (RSA) [20], and Artificial Rabbits Optimization (ARO) [21].

When these optimizers are adapted to cloud task scheduling, the main research problem is no longer only how to improve a generic search equation. The scheduler must also encode a continuous-to-discrete mapping, preserve feasible assignments, and evaluate candidates under the same cost model. Recent cloud-oriented variants reflect this pattern: enhanced DBO scheduling [25], E-AVOA-TS [26], GTO-based fault-tolerant scheduling [27], EARTH with elite strategies and chaotic mapping [28, 29], CEEO for IoT-generated cloud tasks [30], and DECSO for multi-objective cloud scheduling [31]. These works are used here as methodological context rather than as direct experimental baselines.

## 2.3 Problem-Specific Hybridization and Local Search

Hybridization is common in cloud meta-heuristic scheduling, but not every added operator contributes equally. Lévy-enhanced Osprey Optimization [22, 23], chaotic RIME with generalized quadratic interpolation and Lévy flight [32], and hybrid SAO-RIME scheduling [33] show that stochastic perturbations can improve search diversity. The Mantegna scheme provides a standard way to generate stable Lévy-distributed steps [8], and Lévy flight was introduced prominently into Cuckoo Search by Yang and Deb [9].

For cloud scheduling, however, diversity alone is not enough. The reviewed literature suggests that the strongest gains occur when a meta-heuristic is coupled with a mechanism that understands the scheduling structure. This is the role of task-reassignment local search in LSCBO: after a candidate assignment has been decoded, the scheduler tests whether moving tasks from the most-loaded VM toward the least-loaded VM improves the scalarized objective. This follows the memetic principle of coupling a general optimizer with a problem-specific local improvement operator [36].

## 2.4 Positioning and Comparison Boundary of LSCBO

The Coyote-Badger Optimization (CBO) framework is inspired by documented coyote-badger interactions [2] and formalizes the behavior as a three-phase population search process [24]. For the present work, the biological analogy is not treated as evidence of performance. We are aware of the ongoing methodological debate in the optimization community regarding the proliferation of metaphor-based metaheuristics, and we therefore deliberately ground our design choices in CBO’s modular phase structure rather than in its biological narrative. CBO is selected because its searching, encircling, and attacking phases form a modular structure that can be modified and ablated without redesigning the entire optimizer; the component-ablation experiment in Section 4.2.8 exploits exactly this modularity to attribute performance to specific mechanisms rather than to the metaphor.

LSCBO is therefore positioned as a CBO-based scheduler with two controlled additions: Lévy-assisted stochastic exploration and task-reassignment local search. Its direct comparison focuses on population-based meta-heuristic schedulers under identical task, VM, seed, population-size, and iteration settings. Restricting the experimental comparison to other population-based meta-heuristic schedulers follows the established protocol of recent cloud meta-heuristic scheduling studies such as enhanced DBO scheduling [25], E-AVOA-TS [26], EARTH [28, 29], and DECSO [31], which likewise benchmark against the meta-heuristic family rather than against classical dispatching rules. Deterministic dispatching rules such as Min-Min and Max-Min, DRL schedulers such as DeepRM [38] and Decima [37], exact methods, and Pareto-front algorithms remain important adjacent paradigms, but they require different protocols and are therefore treated as methodological context rather than as direct experimental baselines, and are not used as the basis for a universal state-of-the-art claim.

### 3 Research Methodology

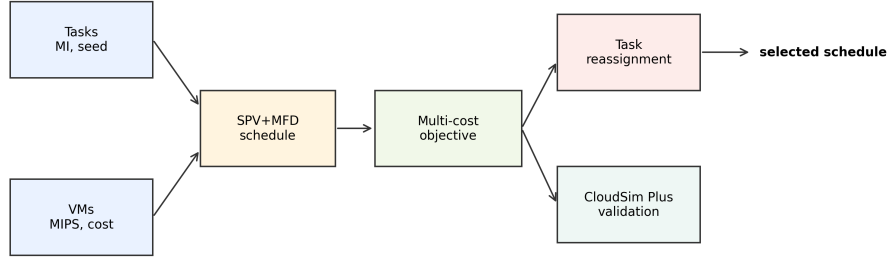
#### 3.1 Multi-Cost Cloud Task Scheduling Model

##### 3.1.1 System Model

The cloud datacenter provides  $M$  heterogeneous Virtual Machines  $V = \{VM_1, \dots, VM_M\}$  processing  $N$  independent tasks  $T = \{T_1, \dots, T_N\}$ . The computational capacity is defined by  $MIPS_j$  for  $VM_j$  and  $MI_i$  size for  $T_i$ . The Expected Time to Compute ( $ETC$ ) is:

$$ETC_{i,j} = \frac{MI_i}{MIPS_j} \quad (1)$$

All  $N$  tasks are assumed to be known a priori (offline batch scheduling), which is standard in the meta-heuristic scheduling literature [1] and appropriate for scenarios such as scientific workflow execution and periodic batch processing. The main notation used throughout this section is summarized in Table 1, and the overall multi-cost scheduling model is illustrated in Fig. 1.



**Fig. 1** Multi-cost cloud task scheduling model used by LSCBO. A continuous population search is decoded into feasible task-to-VM assignments, evaluated by makespan, energy, execution cost, and load-balance criteria, and then improved by task-reassignment local search before the selected schedule is evaluated in the CloudSim Plus experiment pipeline.

##### 3.1.2 Scheduling Criteria and Scalarized Objective

We combine four criteria into a single scalar objective via equal-weight summation:

**Makespan:** Makespan =  $\max_{j \in \{1, \dots, M\}} \{CT_j\}$ , where  $CT_j = \sum_{i=1}^N (ETC_{i,j} \times x_{i,j})$  and  $x_{i,j} \in \{0, 1\}$  indicates task assignment.

**Energy Consumption:** Energy =  $\sum_{j=1}^M \int_0^{T_{ms}} P_j(u_j(t)) dt$ , where  $P_j(u) = P_j^{idle} + (P_j^{max} - P_j^{idle}) \times u$  is the CloudSim linear power model with  $P_j^{max} = 250W$  and  $P_j^{idle} = 0.6 \times P_j^{max}$ .

**Table 1** Main Notation for Cloud Task Scheduling

| Symbol      | Description                                     |
|-------------|-------------------------------------------------|
| $N$         | Number of cloud tasks                           |
| $M$         | Number of virtual machines                      |
| $MI_i$      | Million instructions of task $T_i$              |
| $MIPS_j$    | Processing speed of virtual machine $VM_j$      |
| $ETC_{i,j}$ | Expected execution time of task $T_i$ on $VM_j$ |
| $CT_j$      | Completion time of virtual machine $VM_j$       |
| $x_{i,j}$   | Binary task-assignment variable                 |
| $F$         | Scalarized scheduling objective                 |
| $w_k$       | Weight of the $k$ -th scheduling criterion      |
| $I_{LS}$    | Local-search interval of LSCBO                  |

**Execution Cost:**  $\text{Cost} = \sum_{j=1}^M (C_j \times CT_j)$ , with  $C_j$  proportional to MIPS (0.05–0.20 cost units per unit time).

**Load Balance:**  $LB_\sigma = \sqrt{\frac{1}{M} \sum_{j=1}^M (CT_j - CT_{avg})^2}$ . We also report the normalized load-balance ratio  $LBR = \frac{LB_\sigma}{CT_{avg}}$ .

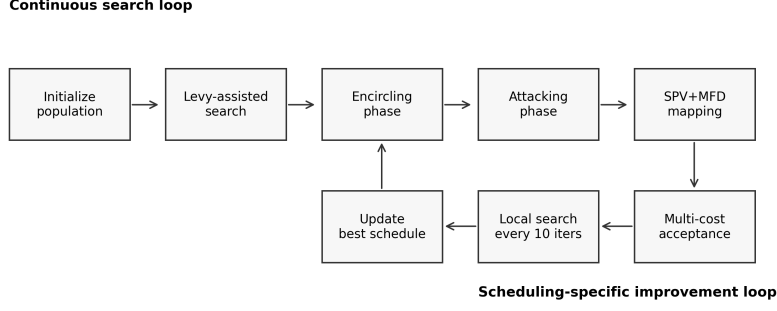
The scalarized objective is  $F = \sum_{k=1}^4 w_k f_k$  with  $\sum w_k = 1$ , where  $f_{ms} = \text{Makespan}/c_{ms}$ ,  $f_{en} = \text{Energy}/c_{en}$ ,  $f_{cost} = \text{Cost}/c_{cost}$ ,  $f_{lb} = \text{LBR}$ , and  $c_{ms}, c_{en}, c_{cost}$  are workload-specific normalization constants. Default experiments use equal weights ( $w_k = 0.25$ ); a comprehensive weight sensitivity analysis across five configurations is presented in Section 4.2.6.

### 3.1.3 Continuous-to-Discrete Scheduling Representation

LSCBO searches in a continuous  $N$ -dimensional space and decodes each candidate vector into a feasible task assignment before evaluation. The Smallest Position Value (SPV) rule sorts task coordinates into a ranked task sequence; the Modulo-Fill Dispatch (MFD) rule then assigns the task at rank  $s_i$  to  $VM_{target} = (s_i \bmod M) + 1$ , distributing ranked tasks across VMs by the modulo of their rank index. This representation keeps every task assigned to exactly one VM while allowing a continuous optimizer to modify the relative task order. After decoding, every candidate is evaluated by the scalarized objective  $F$ , so acceptance depends on the joint multi-cost score rather than on makespan alone.

## 3.2 LSCBO Algorithm Design and Improvements

LSCBO extends the CBO framework with two key enhancements: (1) a greedy task-reassignment local search that exploits the decoded scheduling structure, and (2) Lévy-flight exploration that replaces the original tanh-based searching operator. Phases 2 (rotation-matrix encircling) and 3 (static attack with  $w = 0.5$ ) of the original CBO are preserved unchanged. Figure 2 summarizes how the continuous search, discrete schedule decoding, multi-cost evaluation, and local reassignment loop are connected. The complete LSCBO pseudocode is presented in Algorithm 1.



**Fig. 2** Workflow of the LSCBO scheduler. Lévy-assisted CBO explores continuous task-order vectors; SPV+MFD decoding converts each vector into a feasible schedule; the scalarized objective evaluates makespan, energy, cost, and load balance; and task-reassignment local search periodically improves the current best assignment.

### 3.2.1 Population Initialization

In the CBO framework, the population is modeled on the symbiotic relationship between coyotes and badgers. Let the total population be  $2P$  individuals, partitioned into  $P$  symbiotic pairs. For a  $\Lambda$ -dimensional search space, the initial positions of the  $i$ -th coyote and badger are sampled component-wise within  $[LB, UB]$ :

$$X_{coy,i}^0 = LB + r_{coy,i}^{(0)} \odot (UB - LB), \quad r_{coy,i}^{(0)} \sim \mathcal{U}([0, 1]^\Lambda) \quad (2)$$

$$X_{bad,i}^0 = LB + r_{bad,i}^{(0)} \odot (UB - LB), \quad r_{bad,i}^{(0)} \sim \mathcal{U}([0, 1]^\Lambda) \quad (3)$$

The target prey position for each pair is the midpoint:

$$X_{prey,i}^0 = \frac{X_{coy,i}^0 + X_{bad,i}^0}{2} \quad (4)$$

### 3.2.2 Phase 1: Lévy-Flight Searching

The original CBO searching phase uses a tanh-based deterministic operator. LSCBO replaces this with Lévy-flight exploration, which introduces heavy-tailed stochastic perturbations:

$$X_{coy,i}^{t+1/3} = X_{coy,i}^t + \alpha \odot L_{coy,i}^t(\beta) \quad (5)$$

$$X_{bad,i}^{t+1/3} = X_{bad,i}^t + \alpha \odot L_{bad,i}^t(\beta) \quad (6)$$

where  $\alpha = \alpha_0(1 - t/T)$  with  $\alpha_0 = 0.05$  is a linearly decreasing step size, and  $L(\beta)$  are Lévy-flight step vectors generated by the Mantegna scheme [8] with stability parameter  $\beta = 1.5$ . For each dimension  $d$ :

$$\ell_{i,d} = \frac{u_{i,d}}{|v_{i,d}|^{1/\beta}}, \quad u_{i,d} \sim \mathcal{N}(0, \sigma_u^2), \quad v_{i,d} \sim \mathcal{N}(0, 1) \quad (7)$$



---

**Algorithm 1** LSCBO: Coyote-Badger Optimization with Task-Reassignment Local Search and Lévy Perturbation

---

```

1: Input: Population size  $2P$ , max iterations  $T$ , local search interval  $I_{LS}$ , Lévy
   parameter  $\beta = 1.5$ , step scale  $\alpha_0 = 0.05$ 
2: Initialize coyote population  $\{X_{coy,i}^0\}_{i=1}^P$  and badger population  $\{X_{bad,i}^0\}_{i=1}^P$ 
3: Initialize prey positions  $X_{prey,i}^0$  as the midpoint of each coyote-badger pair
4: Evaluate all individuals; record global best solution  $X^*$ 
5: for  $t = 1$  to  $T$  do
6:    $\alpha \leftarrow \alpha_0(1 - t/T)$  ▷ Decreasing step size
7:   for each symbiotic pair  $i = 1$  to  $P$  do
8:     Phase 1 – Lévy Searching: Update  $X_{coy,i}^t$  and  $X_{bad,i}^t$  using Mantegna
       Lévy steps
9:     Apply boundary projection; decode via SPV+MFD; greedy accept if
       improved
10:    Phase 2 – Encircling: Apply rotation matrix  $R_{p,q}(\theta)$ 
11:    Decode via SPV+MFD; greedy accept if improved
12:    Phase 3 – Attacking: Update toward leaders  $\Omega_{coy}^t, \Omega_{bad}^t$ 
13:    Decode via SPV+MFD; greedy accept if improved
14:  end for
15:  if  $t \bmod I_{LS} = 0$  then
16:    Task-Reassignment Local Search: Apply greedy task reassignment to
       $X^*$  (Section 3.2.5)
17:    Update  $X^*$  if improved
18:  end if
19:  Update global leaders  $\Omega_{coy}^t, \Omega_{bad}^t$ 
20: end for
21: Output: Best schedule  $X^*$ 

```

---

$$\sigma_u = \left[ \frac{\Gamma(1 + \beta) \sin(\pi\beta/2)}{\Gamma((1 + \beta)/2) \beta 2^{(\beta-1)/2}} \right]^{1/\beta} \quad (8)$$

The Lévy distribution exhibits heavy-tailed power-law decay  $L(s) \sim |s|^{-1-\beta}$ , enabling occasional long-range jumps that enhance global exploration. The linearly decreasing  $\alpha$  ensures that exploration is gradually reduced as the search progresses, allowing later iterations to focus on exploitation.

### 3.2.3 Phase 2: Encircling

Phase 2 of CBO is preserved unchanged. Individuals adjust their approach angles using a Givens-type rotation matrix acting on a randomly selected coordinate pair  $(p, q)$ :

$$\tilde{X}_i^{rot} = X_{prey,i}^t + R_{p,q}(\theta) (X_i^t - X_{prey,i}^t) \quad (9)$$

where  $R_{p,q}(\theta)$  is the identity matrix except for a  $2 \times 2$  rotation block, and  $\theta$  decreases linearly from  $\theta_0 = \pi$  to zero. A greedy selection rule retains improved candidates.

### 3.2.4 Phase 3: Static Attacking

Phase 3 of CBO is preserved unchanged. Individuals are updated toward the current best coyote and badger leaders  $\Omega_{coy}^t$  and  $\Omega_{bad}^t$ :

$$X_{coy,i}^{t+1} = X_{coy,i}^t + 0.5 \times (\Omega_{coy}^t - X_{coy,i}^t) \quad (10)$$

$$X_{bad,i}^{t+1} = X_{bad,i}^t + 0.5 \times (\Omega_{bad}^t - X_{bad,i}^t) \quad (11)$$

The fixed attack weight  $w = 0.5$  is preserved from the original CBO design so that LSCBO modifies the search framework only through the stated Lévy-assisted search and task-reassignment local search mechanisms.

### 3.2.5 Task-Reassignment Local Search

Every  $I_{LS} = 10$  iterations, LSCBO applies a greedy task-reassignment local search to the current global best solution. The procedure is:

1. Identify the most-loaded VM ( $VM_{worst}$ ) and the least-loaded VM ( $VM_{best}$ ) based on current completion times.
2. For each task  $i$  currently assigned to  $VM_{worst}$ , compute the scalarized objective of the candidate schedule obtained by reassigning  $i$  to  $VM_{best}$ .
3. Accept the reassignment that maximally reduces the scalarized objective.
4. Repeat up to  $\min(10, \text{tasks on } VM_{worst})$  swaps per call.

This local search directly exploits the scheduling problem structure—a design principle aligned with memetic algorithms [36]. The operator performs greedy load balancing by redistributing tasks from over-utilized VMs to under-utilized VMs before execution. Candidate reassignments are accepted only if they improve the complete scalarized objective  $F$  (not makespan alone), ensuring balanced optimization across all criteria used by the scheduler.

After every update, each coordinate is projected back to the feasible interval via  $X_{i,d}^{t+1} \leftarrow \min(UB_d, \max(LB_d, X_{i,d}^{t+1}))$ . For cloud scheduling, the projected vector is decoded via SPV+MFD and evaluated through the scalarized objective (Section 3.1); the new position is greedily accepted only if it improves the current objective value.

### 3.2.6 Evolution-Path Analysis

To isolate the contribution of each design decision, we conducted a 5-stage evolution-path experiment (750 total trials, 5 stages  $\times$  5 task scales  $\times$  30 seeds) tracing the development from CBO to the local-search-enhanced variant. Table 2 reports both makespan and the scalarized objective, because the scheduler accepts candidate solutions using the complete multi-cost score rather than makespan alone.

The evolution-path analysis shows that the largest improvement occurs when task-reassignment local search is introduced (V2 $\rightarrow$ V3, +15.4% in objective). Lévy flight alone provides only a small improvement in this diagnostic path, and adding Lévy to an already local-search-enhanced variant does not consistently improve the scalarized objective. The final LSCBO design is therefore interpreted as a CBO-based scheduler whose main scheduling-specific mechanism is task reassignment, with Lévy perturbation

**Table 2** Evolution Path from CBO to LSCBO (30 seeds, all task scales)

| Stage                         | Makespan     | Objective    | Objective change |
|-------------------------------|--------------|--------------|------------------|
| V1: CBO                       | 618.0        | 1.659        | —                |
| V2: CBO + Lévy                | 615.4        | 1.651        | +0.5%            |
| V3: CBO + local search        | 533.0        | 1.397        | +15.4%           |
| V4: CBO + Lévy + local search | 534.1        | 1.401        | −0.3%            |
| <b>V5: final LSCBO</b>        | <b>534.4</b> | <b>1.394</b> | —                |

Note: Objective change is computed against the preceding row, where positive values indicate improvement. The largest within-path improvement occurs when task-reassignment local search is introduced. Stages V4 and V5 use the identical final LSCBO configuration (CBO with Lévy-assisted search and task-reassignment local search); V5 is a confirmatory repeat, so its objective-change entry is shown as “—” because no design change is introduced relative to V4, and the small numerical V4→V5 difference reflects only stochastic run-to-run variation.

retained as an auxiliary diversity mechanism rather than as the primary source of improvement.

### 3.3 LSCBO-Based Cloud Task Scheduling

Deploying LSCBO as a cloud task scheduler couples its continuous search with the multi-cost scheduling model and the SPV+MFD decoding introduced earlier in this section. At every iteration, each candidate position vector is decoded into a feasible task-to-VM assignment, scored by the scalarized multi-cost objective, and accepted greedily when it improves the incumbent; every  $I_{LS}$  iterations the task-reassignment local search refines the current best schedule. Figure 2 summarizes this end-to-end pipeline and Algorithm 1 states the complete procedure.

#### 3.3.1 Computational Complexity

Let  $S = 2P$  denote the total number of search agents,  $D$  the continuous search dimension,  $T$  the maximum number of iterations,  $N$  the number of tasks, and  $M$  the number of VMs. The three-phase per-iteration update requires  $O(SD)$  operations per phase, yielding  $O(TSD)$  overall for continuous benchmarks. For cloud scheduling, SPV+MFD decoding adds  $O(N \log N + M)$  per candidate, giving  $O(TS(N \log N + M))$  total under straightforward decoding. The task-reassignment local search is invoked every  $I_{LS} = 10$  iterations; each call evaluates at most  $L = \min(10, n_{worst})$  reassignment candidates from the most-loaded VM. With direct recomputation, one local-search call costs  $O(L(N + M))$ , and the total local-search overhead is  $O(\lfloor T/I_{LS} \rfloor L(N + M))$ . In the CloudSim experiments this overhead is bounded by at most 20 additional schedule evaluations per run.

## 4 Experimental Results

The performance of LSCBO is evaluated through two experimental series. Series 1 validates the algorithm’s global optimization capability on 29 CEC2017 benchmark functions against six comparison algorithms. Series 2 evaluates LSCBO in the cloud task scheduling domain using a CloudSim Plus scheduling pipeline—in which every formal

row is executed through a full CloudSim Plus simulation—against eight population-based meta-heuristic schedulers across comprehensive experimental campaigns.

To ensure a fair comparison, the comparison algorithms retain their standard control parameters where applicable, while population size, iteration count, task instances, VM instances, and random seeds are fixed within each experimental series. Table 3 summarizes the settings for the CEC2017 experiments (Series 1), and Table 5 for the cloud scheduling experiments (Series 2).

## 4.1 Experimental Series 1: CEC2017 Global Optimization Benchmarks

To validate LSCBO’s fundamental optimization capability before applying it to the scheduling domain, we evaluate the algorithm on the CEC2017 benchmark suite, which comprises rotated and shifted test functions of varying complexity and modality. This two-tier evaluation strategy—benchmark validation followed by domain application—follows the established methodology in meta-heuristic research [10, 11, 24]. This series is intended as a general-capability sanity check rather than as a novelty claim in continuous optimization: its purpose is to confirm that the Lévy-modified search does not degrade the base optimizer before the algorithm is applied to its primary target domain, cloud task scheduling. Achieving competitive (rather than uniquely dominant) continuous-optimization performance is sufficient for this purpose.

### 4.1.1 CEC2017 Settings

All CEC2017 experiments use a population size of 30 with 10,000 iterations (300,000 function evaluations per run), with  $D = 30$  and search bounds  $[-100, 100]^D$ . We selected 29 functions (F1, F3–F30; F2 is deprecated) from the CEC2017 suite [50].

### 4.1.2 CEC2017 Ranking Comparison

We compare LSCBO against six algorithms on all 29 CEC2017 functions: CBO [24], GTO [34], DE [6], PSO [4], SA [49], and GWO [35]. This selection encompasses both classical and more recent optimizers, providing a balanced reference for convergence speed, solution quality, and stability under an identical evaluation budget. The two experimental series intentionally use different comparison sets: the CEC2017 series includes the standard continuous-optimization references DE and SA, which are conventional baselines for unconstrained numerical benchmarking, whereas the cloud-scheduling series (Section 4.2) substitutes the recent population-based optimizers WOA, HHO, DBO, and AOA, which are the more representative meta-heuristic baselines in the cloud task-scheduling literature. The shared anchors CBO, GTO, PSO, and GWO appear in both series to maintain a consistent reference across the two settings. All algorithms are evaluated over 300,000 function evaluations (population size  $30 \times 10,000$  iterations) per run, with  $D = 30$  and search bounds  $[-100, 100]^D$ . Table 4 presents the Friedman average ranking across all 29 functions, and Fig. 3 visualizes the same ranking.

**Statistical Analysis.** A Friedman test over the 29 CEC2017 functions (functions as blocks, algorithms as treatments) confirms significant differences among algorithms ( $\chi^2 = 156.04$ ,  $p < 0.0001$ ). Post-hoc Nemenyi analysis [45] yields a critical difference

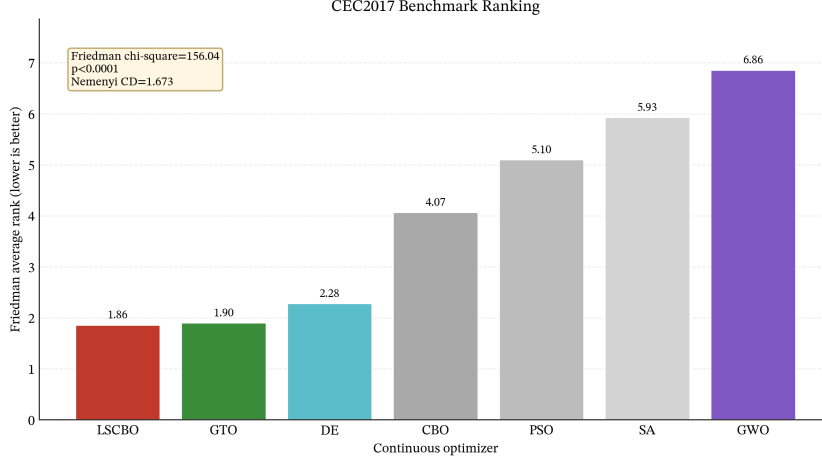
**Table 3** Parameter Settings for CEC2017 Experiments (Series 1)

| Algorithm                                                                             | Parameter                   | Value                 |
|---------------------------------------------------------------------------------------|-----------------------------|-----------------------|
| LSCBO                                                                                 | Population size             | 30                    |
|                                                                                       | Iterations                  | 10,000                |
|                                                                                       | Lévy stability $\beta$      | 1.5                   |
|                                                                                       | Step scale $\alpha_0$       | 0.05                  |
| CBO                                                                                   | Population size             | 30                    |
|                                                                                       | Iterations                  | 10,000                |
|                                                                                       | Rotation range $\theta_0$   | $\pi$                 |
|                                                                                       | Attack weight $w$           | 0.5                   |
| GTO                                                                                   | Population size             | 30                    |
|                                                                                       | Migration rate $p$          | 0.03                  |
|                                                                                       | Silverback factor $\beta$   | 3.0                   |
| DE                                                                                    | Population size             | 30                    |
|                                                                                       | Scale factor $F$            | 0.5                   |
|                                                                                       | Crossover rate $CR$         | 0.9                   |
| PSO                                                                                   | Population size             | 30                    |
|                                                                                       | Inertia weight $w$          | $0.9 \rightarrow 0.4$ |
|                                                                                       | Cognitive/social $c_1, c_2$ | 2.0, 2.0              |
| SA                                                                                    | Initial temperature $T_0$   | 1,000                 |
|                                                                                       | Cooling rate $\gamma$       | 0.95                  |
| GWO                                                                                   | Population size             | 30                    |
|                                                                                       | Convergence constant $a$    | $2 \rightarrow 0$     |
| All algorithms: $D = 30$ , bounds $[-100, 100]^D$ , 30 independent runs per function. |                             |                       |

**Table 4** Friedman Ranking of Algorithms on CEC2017 (29 Functions, 30 Runs per Function)

| Algorithm | Average Rank |
|-----------|--------------|
| LSCBO     | 1.86         |
| GTO       | 1.90         |
| DE        | 2.28         |
| CBO       | 4.07         |
| PSO       | 5.10         |
| SA        | 5.93         |
| GWO       | 6.86         |

CD = 1.673 at  $\alpha = 0.05$ . LSCBO, GTO, and DE form the top cluster with pairwise rank gaps within the CD, indicating competitive continuous search performance among these three algorithms. Pairwise conclusions are further examined with Wilcoxon signed-rank tests [47]. LSCBO achieves statistically significant wins over CBO, PSO, SA, and GWO ( $p < 0.001$  for each), and statistical parity with DE ( $p = 0.059$ ) and GTO ( $p = 0.481$ ). It should be noted that the comparison set does not include specialized continuous solvers such as L-SHADE [43] or CMA-ES [44], and the primary evaluation target of



**Fig. 3** Friedman average ranking of seven algorithms on 29 CEC2017 benchmark functions. LSCBO obtains the lowest average rank (1.86) within this comparison set, followed by GTO (1.90) and DE (2.28). The global Friedman test confirms significant differences ( $\chi^2 = 156.04$ ,  $p < 0.0001$ ).

LSCBO is discrete cloud scheduling (Section 4.2). Per-function average fitness values and standard deviations for all algorithms are available in the Supplementary Materials.

#### 4.1.3 Convergence Analysis on CEC2017

Convergence curves on representative CEC2017 functions demonstrate that LSCBO exhibits rapid early convergence and continues to improve in later iterations. The algorithm reaches highly competitive final fitness values compared with the tested baselines. Complete per-function convergence data is available in the Supplementary Materials and project repository.

## 4.2 Experimental Series 2: Cloud Task Scheduling

### 4.2.1 CloudSim Settings

All cloud-scheduling meta-heuristic experiments use a population size of 20 with 20 iterations, corresponding to 20 initial evaluations and 400 iteration evaluations per run. This compact budget reflects the time-constrained nature of batch scheduling decisions, where each candidate evaluation entails a full schedule decoding and cost computation; it is applied identically to all compared schedulers so that the comparison remains fair under a common evaluation budget. Because every scheduler operates under the same modest iteration count, the reported differences characterize the proposed CBO-plus-local-search design pattern within this shared protocol rather than asymptotic search behavior under very large budgets, which is left to future work. For LSCBO, the local search invoked every  $I_{LS} = 10$  iterations performs at most 10 greedy task-reassignment attempts per call. These local-search attempts are counted separately: the worst-case LSCBO budget is therefore 440 total schedule evaluations including

initialization, compared with 420 for the other schedulers. This approximately 4.8% overhead is reported explicitly rather than treated as an identical evaluation budget.

**Table 5** Parameter Settings for CloudSim Experiments (Series 2)

| Algorithm                                                                                                                                                                                                     | Parameter                       | Value                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|-----------------------|
| LSCBO                                                                                                                                                                                                         | Population size                 | 20                    |
|                                                                                                                                                                                                               | Iterations                      | 20                    |
|                                                                                                                                                                                                               | Lévy stability $\beta$          | 1.5                   |
|                                                                                                                                                                                                               | Step scale $\alpha_0$           | 0.05                  |
|                                                                                                                                                                                                               | Local search interval $I_{LS}$  | 10                    |
|                                                                                                                                                                                                               | Attack weight $w$               | 0.5                   |
| CBO                                                                                                                                                                                                           | Population size                 | 20                    |
|                                                                                                                                                                                                               | Iterations                      | 20                    |
|                                                                                                                                                                                                               | Rotation range $\theta_0$       | $\pi$                 |
|                                                                                                                                                                                                               | Attack weight $w$               | 0.5                   |
| GTO                                                                                                                                                                                                           | Population size                 | 20                    |
|                                                                                                                                                                                                               | Iterations                      | 20                    |
|                                                                                                                                                                                                               | Migration probability $p$       | 0.03                  |
| WOA, GWO                                                                                                                                                                                                      | Population size                 | 20                    |
|                                                                                                                                                                                                               | Iterations                      | 20                    |
|                                                                                                                                                                                                               | Convergence constant $a$        | $2 \rightarrow 0$     |
| HHO                                                                                                                                                                                                           | Population size                 | 20                    |
|                                                                                                                                                                                                               | Iterations                      | 20                    |
| PSO                                                                                                                                                                                                           | Population size                 | 20                    |
|                                                                                                                                                                                                               | Iterations                      | 20                    |
|                                                                                                                                                                                                               | Inertia weight $w$              | $0.9 \rightarrow 0.4$ |
|                                                                                                                                                                                                               | Cognitive/social $c_1, c_2$     | 2.0, 2.0              |
| DBO                                                                                                                                                                                                           | Population size                 | 20                    |
|                                                                                                                                                                                                               | Iterations                      | 20                    |
|                                                                                                                                                                                                               | Guide probability $P_{percent}$ | 0.2                   |
| AOA                                                                                                                                                                                                           | Population size                 | 20                    |
|                                                                                                                                                                                                               | Iterations                      | 20                    |
|                                                                                                                                                                                                               | Sensitive parameter $\alpha$    | 5.0                   |
| All CloudSim algorithms are evaluated on identical task instances, VM instances, seeds, population sizes, and iteration counts. LSCBO additionally reports the counted local-search overhead described above. |                                 |                       |

#### 4.2.2 CloudSim Configuration

The scheduling implementation is built around CloudSim Plus 8.0.0 [41, 42]. The simulated environment comprises one datacenter with at least 10 hosts; each host has 4 CPU cores, 16 GB RAM, 1 TB storage, and 10 Gbps bandwidth, and the host count is expanded when needed to allocate all VMs. TimeShared policies are used for both hosts and VMs. All experiments use  $M = 50$  heterogeneous VMs (MIPS 500–2000, uniformly sampled), task MI ranging from 1000 to 20000 (uniformly sampled), and

deterministic seeds 43–72 for reproducibility (30 independent runs). The full formal comparison contains 12,780 multi-cost scheduling rows generated from this task/VM model. Every one of these 12,780 rows was additionally executed through a full CloudSim Plus simulation: all scheduled cloudlets completed successfully (12,780/12,780 rows validated), and the CloudSim-measured makespan tracks the analytic multi-cost values reported in this section closely, confirming that the reported schedules are executable and faithfully simulated rather than only analytically scored. The complete implementation is publicly available at <https://github.com/Lazywords2006/LSCBO>.

### 4.2.3 Main Comparison of Meta-Heuristic Schedulers

Table 6 presents the makespan comparison across five task scales. All values are the mean of 30 independent runs under fixed seeds (43–72).

**Table 6** Cross-Scale Makespan Comparison (mean, 30 runs per scale)

| Algorithm      | $N = 200$   | $N = 500$    | $N = 1000$   | $N = 2000$   | $N = 5000$    |
|----------------|-------------|--------------|--------------|--------------|---------------|
| <b>LSCBO</b>   | <b>46.9</b> | <b>121.4</b> | <b>275.7</b> | <b>601.3</b> | <b>1620.5</b> |
| WOA            | 67.8        | 171.1        | 342.8        | 699.9        | 1794.1        |
| HHO            | 67.1        | 172.0        | 344.7        | 701.6        | 1785.8        |
| CBO (baseline) | 67.0        | 171.2        | 344.6        | 705.0        | 1792.6        |
| PSO            | 68.5        | 170.3        | 344.1        | 703.1        | 1798.5        |
| DBO            | 67.7        | 171.9        | 345.6        | 701.1        | 1797.9        |
| GTO            | 68.5        | 170.6        | 347.3        | 702.2        | 1790.6        |
| GWO            | 68.7        | 171.7        | 347.0        | 702.0        | 1800.3        |
| AOA            | 68.5        | 172.2        | 345.6        | 702.0        | 1803.1        |

Table 7 presents the load-balance ratio (LBR, lower is better).

**Table 7** Cross-Scale Load-Balance Ratio (LBR) Comparison

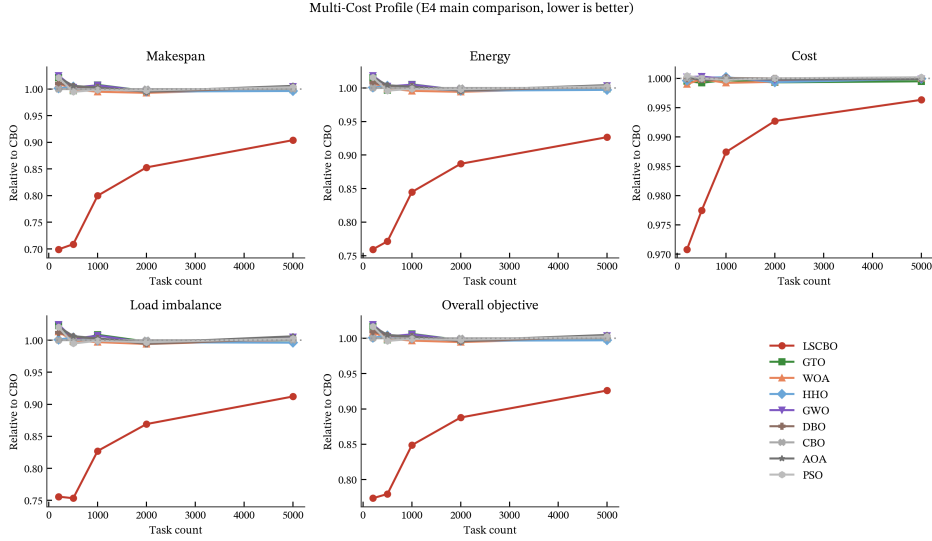
| Algorithm    | $N = 200$    | $N = 500$    | $N = 1000$   | $N = 2000$   | $N = 5000$   |
|--------------|--------------|--------------|--------------|--------------|--------------|
| <b>LSCBO</b> | <b>1.359</b> | <b>1.355</b> | <b>1.492</b> | <b>1.597</b> | <b>1.709</b> |
| WOA          | 1.821        | 1.797        | 1.799        | 1.827        | 1.875        |
| HHO          | 1.800        | 1.806        | 1.804        | 1.832        | 1.867        |
| CBO          | 1.798        | 1.799        | 1.804        | 1.838        | 1.874        |
| PSO          | 1.834        | 1.789        | 1.803        | 1.832        | 1.879        |
| DBO          | 1.819        | 1.808        | 1.809        | 1.827        | 1.878        |
| GTO          | 1.838        | 1.796        | 1.820        | 1.834        | 1.874        |
| GWO          | 1.842        | 1.802        | 1.817        | 1.831        | 1.882        |
| AOA          | 1.836        | 1.810        | 1.810        | 1.831        | 1.885        |



Across the 30-seed formal comparison, LSCBO achieves the lowest mean makespan and LBR at all five task scales. Table 8 reports the corresponding multi-cost improvements over the standardized CBO baseline. The improvement is largest at smaller task scales and remains positive at  $N = 5000$ , where the search space is larger and the relative gap naturally narrows.

**Table 8** Per-Scale LSCBO Improvement over CBO by Cost Component

| Scale      | Makespan | Energy | Cost | LBR   | Objective |
|------------|----------|--------|------|-------|-----------|
| $N = 200$  | 30.1%    | 24.1%  | 2.9% | 24.4% | 22.6%     |
| $N = 500$  | 29.1%    | 22.9%  | 2.3% | 24.7% | 22.0%     |
| $N = 1000$ | 20.0%    | 15.5%  | 1.3% | 17.3% | 15.1%     |
| $N = 2000$ | 14.7%    | 11.3%  | 0.7% | 13.1% | 11.2%     |
| $N = 5000$ | 9.6%     | 7.3%   | 0.4% | 8.8%  | 7.4%      |

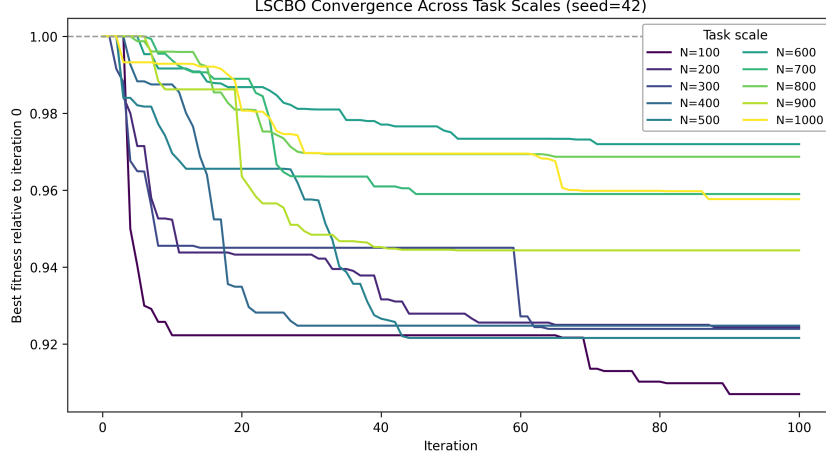


**Fig. 4** Formal multi-cost profile aligned with the E4 main-comparison setup. The figure uses the SPV+MFD decoding, power/cost parameters, five task scales, nine algorithms, and 30 paired seeds; each panel reports the mean value relative to CBO at the same task scale, where values below 1.0 indicate improvement over the standardized CBO baseline.

Figure 4 complements the makespan and LBR tables by exposing the cost components used by the scalarized objective. It shows that the strongest effects appear in makespan, energy, and load balance, while cost improves more modestly because all methods execute the same task set on the same VM-price model. This connects the algorithmic acceptance rule with the cloud-scheduling model in Section 3.1.

#### 4.2.4 Convergence Analysis

Representative convergence traces are used as a diagnostic view of search behavior rather than as the primary statistical evidence. Figure 5 shows stored LSCBO scheduling traces across task scales ( $N = 100$  to  $N = 1000$ ) under a fixed seed, with each curve normalized to its own iteration-0 value to expose within-run improvement on a common scale. The traces illustrate how LSCBO’s candidate quality evolves as the problem size grows under the same iteration budget; the comparative conclusion against the baselines is based on the paired repeated-run tables and Friedman/Wilcoxon tests rather than on a single trace.

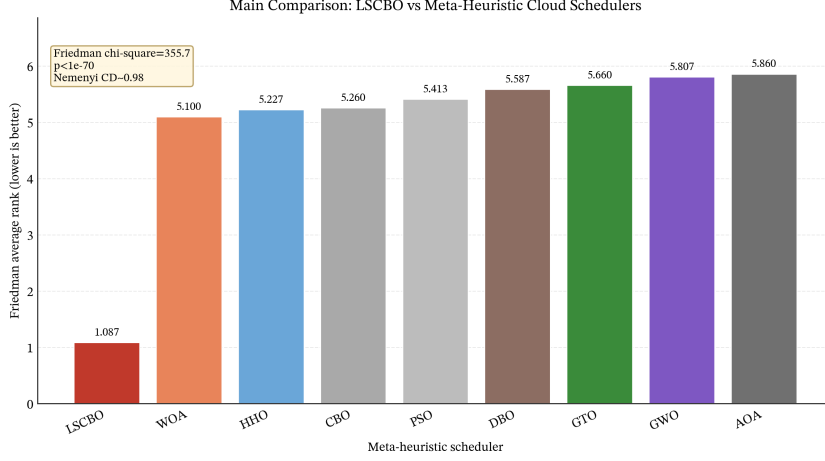


**Fig. 5** Diagnostic convergence traces of LSCBO across task scales ( $N = 100$ – $1000$ ) under a fixed seed. Each curve is normalized by its own initial best fitness to show relative within-run improvement on a common scale; this figure is illustrative of LSCBO’s search behavior and is not used as the primary statistical evidence, which comes from the paired repeated-run tables and Friedman/Wilcoxon tests.

#### 4.2.5 Friedman Ranking and Statistical Tests

The global Friedman test [46] is applied over  $n = 150$  blocks ( $5$  task scales  $\times 30$  paired seeds), confirming significant differences among algorithms ( $\chi^2 = 355.7$ ,  $p < 10^{-70}$ ); the resulting average objective ranks are shown in Fig. 6. The Iman-Davenport correction [48] yields  $F_F \approx 62.8 > F_{0.05}(8, 1192) \approx 1.95$ . LSCBO’s average objective rank of 1.087 is the lowest within the compared population-based meta-heuristic scheduler family. Table 9 presents pairwise Wilcoxon signed-rank tests [47] with Holm-Bonferroni correction.

All pairwise comparisons are statistically significant after Holm-Bonferroni correction (all  $p_{\text{Holm}} < 10^{-24}$ ). The paired win counts show that LSCBO improves the scalarized objective in at least 148 of the 150 matched scale–seed blocks against every opponent. Figure 7 shows the corresponding cross-scale makespan trends for all nine schedulers.



**Fig. 6** Friedman average objective rank across 9 compared meta-heuristic schedulers (150 blocks: 5 scales  $\times$  30 paired seeds). LSCBO obtains the lowest average rank (1.087) within this comparison set (Friedman  $\chi^2 = 355.7$ ,  $p < 10^{-70}$ , Nemenyi CD  $\approx 0.98$  at  $\alpha = 0.05$ ).

**Table 9** Pairwise Wilcoxon Test on Scalarized Objective (LSCBO vs. Each Opponent)

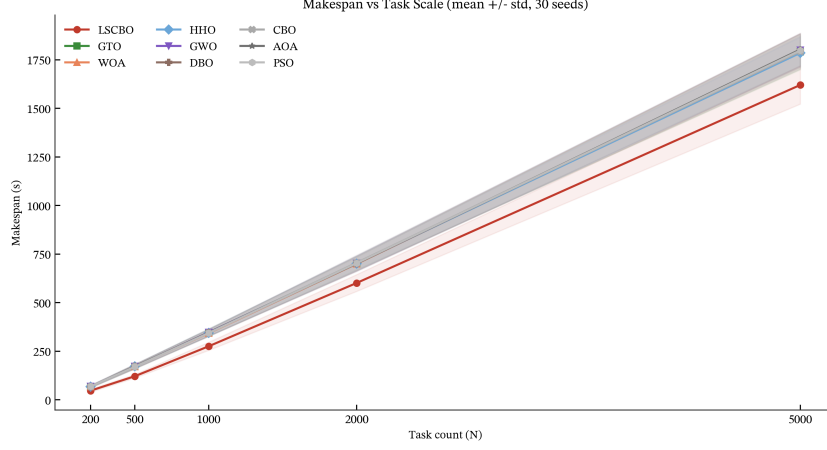
| Opponent       | Objective Impr. | $p$ (raw)              | $p$ (Holm)             | Wins    |
|----------------|-----------------|------------------------|------------------------|---------|
| WOA            | 15.5%           | $2.54 \times 10^{-26}$ | $1.95 \times 10^{-25}$ | 149/150 |
| HHO            | 15.5%           | $3.58 \times 10^{-26}$ | $1.95 \times 10^{-25}$ | 149/150 |
| CBO (baseline) | 15.6%           | $2.44 \times 10^{-26}$ | $1.95 \times 10^{-25}$ | 148/150 |
| PSO            | 15.7%           | $2.54 \times 10^{-26}$ | $1.95 \times 10^{-25}$ | 148/150 |
| DBO            | 15.8%           | $2.59 \times 10^{-26}$ | $1.95 \times 10^{-25}$ | 149/150 |
| GTO            | 15.8%           | $2.65 \times 10^{-26}$ | $1.95 \times 10^{-25}$ | 148/150 |
| GWO            | 16.0%           | $2.44 \times 10^{-26}$ | $1.95 \times 10^{-25}$ | 148/150 |
| AOA            | 16.0%           | $2.44 \times 10^{-26}$ | $1.95 \times 10^{-25}$ | 148/150 |

Two-sided Wilcoxon signed-rank test over the 150 matched scale-seed blocks; raw  $p$ -values are Holm-Bonferroni corrected across the eight comparisons. “Wins” counts the blocks in which LSCBO attains the lower scalarized objective.

#### 4.2.6 Weight Sensitivity Analysis

To empirically test ranking robustness under alternative scalarization preferences, we conducted a comprehensive weight sensitivity experiment across five configurations (Table 10). The experiment includes the population-based meta-heuristic schedulers across 5 task scales ( $N = 100, 300, 500, 800, 1000$ ) with 30 independent runs each.

**Key findings.** LSCBO obtains the best mean scalarized objective under all five weight configurations. This confirms that the primary ranking result is not an artifact of the equal-weight choice. The formal data show the same qualitative ordering when makespan, energy, cost, or load balance is emphasized, with the strongest gains appearing when scheduling pressure is tied to completion time, energy, or load balance.



**Fig. 7** Cross-scale makespan trends for all 9 compared meta-heuristic schedulers in the formal 30-seed data. LSCBO maintains the lowest mean makespan across the tested range  $N = 200$ –5000.

**Table 10** Weight Configurations for Sensitivity Analysis

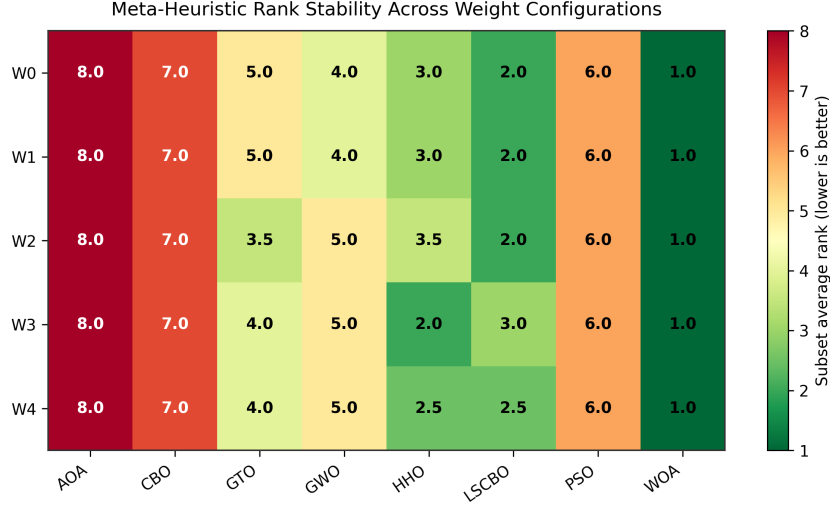
| Config | $w_{ms}$ | $w_{en}$ | $w_{cost}$ | $w_{lb}$ | Description             |
|--------|----------|----------|------------|----------|-------------------------|
| W0     | 0.25     | 0.25     | 0.25       | 0.25     | Equal-weight (baseline) |
| W1     | 0.60     | 0.10     | 0.10       | 0.20     | Makespan-dominant       |
| W2     | 0.10     | 0.40     | 0.40       | 0.10     | Energy-cost-balanced    |
| W3     | 0.85     | 0.05     | 0.05       | 0.05     | Pure makespan           |
| W4     | 0.05     | 0.05     | 0.05       | 0.85     | Pure load-balance       |

The source values used to generate Fig. 8 are stored with the formal full-run CSV files to keep the figure reproducible.

#### 4.2.7 Perturbation Mechanism Comparison

We compared four perturbation mechanisms within the LSCBO framework: (1) Lévy flight ( $\beta = 1.5$ ), (2) Gaussian perturbation, (3) Cauchy perturbation, and (4) random restart ( $p = 0.05$ ), together with a no-perturbation baseline. All configurations use the same task scales and seed set (450 trials: 30 seeds  $\times$  3 task scales  $\times$  5 variants).

The perturbation comparison does not support a strong claim that Lévy flight is independently superior to other perturbation choices. Random restart and even the no-perturbation baseline are numerically close to the Lévy setting in this diagnostic experiment. Therefore, Lévy flight is retained in LSCBO as an auxiliary exploration mechanism, while the paper’s main empirical claim is assigned to task-reassignment local search.



**Fig. 8** Objective-rank heatmap across 5 weight configurations in the formal 30-seed sensitivity data. LSCBO obtains rank 1 under every tested scalarization profile.

**Table 11** Perturbation Mechanism Comparison within the LSCBO Framework

| Variant        | Makespan | Objective | Objective order |
|----------------|----------|-----------|-----------------|
| Random restart | 333.3    | 1.390     | 1               |
| None           | 331.5    | 1.392     | 2               |
| Lévy           | 333.9    | 1.394     | 3               |
| Cauchy         | 335.1    | 1.399     | 4               |
| Gaussian       | 333.5    | 1.402     | 5               |

#### 4.2.8 Phase 2–3 Isolation Ablation

We conducted a targeted ablation experiment using 1050 recorded runs (7 variants  $\times$  5 task scales  $\times$  30 seeds). The table below reports the paired means over 150 matched scale–seed blocks per variant, using the complete LSCBO variant as the reference.

These ablation results show the clearest degradation when task-reassignment local search is removed: the objective increases by 16.0% relative to the complete LSCBO variant, and the CBO base variant is similarly worse. By contrast, the no-Lévy, no-Phase 2, and no-Phase 3 diagnostics are numerically close to, or slightly better than, the complete variant. This result reinforces two points: task reassignment is the robust scheduling-specific contributor, and the remaining CBO-phase and perturbation components interact in a non-additive way rather than producing monotonic gains when each is included.

**Table 12** Formal Component-Ablation Results (vs. Complete LSCBO Variant)

| Variant          | Makespan | Objective | $\Delta$ Objective | Interpretation            |
|------------------|----------|-----------|--------------------|---------------------------|
| LSCBO (complete) | 534.8    | 1.415     | —                  | final design              |
| LS-only          | 545.3    | 1.419     | +0.3%              | local search retained     |
| No Lévy          | 534.0    | 1.402     | −0.9%              | perturbation not decisive |
| No local search  | 613.4    | 1.642     | +16.0%             | local search removed      |
| CBO base         | 615.2    | 1.653     | +16.9%             | no LSCBO additions        |
| No Phase 2       | 532.1    | 1.392     | −1.6%              | interaction effect        |
| No Phase 3       | 536.6    | 1.397     | −1.3%              | interaction effect        |

#### 4.2.9 VM Heterogeneity Robustness

We evaluated the meta-heuristic schedulers across 3 VM configurations (MIPS ratios: 1.5:1, 5:1, and 20:1) at 3 task scales. This experiment is used as a robustness check for the meta-heuristic comparison, not as a claim of dominance over deterministic or learning-based scheduling paradigms.

**Table 13** Mean Objective Across VM Heterogeneity Configurations

| Algorithm    | 1.5:1        | 5:1          | 20:1         |
|--------------|--------------|--------------|--------------|
| <b>LSCBO</b> | <b>1.065</b> | <b>1.530</b> | <b>2.673</b> |
| HHO          | 1.159        | 1.844        | 3.908        |
| WOA          | 1.160        | 1.841        | 3.929        |
| CBO          | 1.159        | 1.846        | 3.931        |
| PSO          | 1.159        | 1.846        | 3.941        |
| GWO          | 1.162        | 1.853        | 3.950        |
| DBO          | 1.162        | 1.849        | 3.953        |
| AOA          | 1.161        | 1.847        | 3.960        |
| GTO          | 1.159        | 1.848        | 3.964        |

Lower values indicate better objective. All nine compared meta-heuristic schedulers are shown; each entry is the mean over the heterogeneity protocol (3 task scales  $\times$  30 seeds = 90 runs per cell).

LSCBO obtains the lowest mean objective under all three VM heterogeneity settings. Its relative advantage is smallest under the near-homogeneous 1.5:1 setting and largest under the 20:1 setting, which is consistent with a scheduling operator that explicitly targets imbalance among heterogeneous VMs.

## 5 Discussion

### 5.1 Performance Analysis

LSCBO demonstrates a consistent and context-dependent performance profile across the two experimental series. On CEC2017 benchmarks, LSCBO obtains the lowest

average rank of 1.86 within the seven-algorithm comparison, with statistically significant wins over CBO, PSO, SA, and GWO ( $p < 0.001$ ) and statistical parity with DE ( $p = 0.059$ ) and GTO ( $p = 0.481$ ), providing a continuous optimization foundation for the scheduling-domain evaluation. In the scheduling domain, LSCBO obtains an average objective rank of 1.087 within the compared population-based meta-heuristic scheduler family, driven primarily by task-reassignment local search—an exploitation mechanism that is specific to task-to-VM assignment.

The multi-cost profile in Fig. 4 clarifies why the local-search mechanism is meaningful for cloud scheduling. Reassigning tasks from heavily loaded VMs can lower the completion-time bottleneck, reduce load imbalance, and indirectly affect the energy and price terms induced by VM utilization. The asymmetric contribution of LSCBO’s two enhancements is therefore instructive. In the internal evolution-path diagnostic experiment, inserting task-reassignment local search produces the largest within-path improvement (V2→V3, +15.4% in objective), and the component-ablation experiment shows that removing local search increases the objective by 16.0% relative to the complete LSCBO variant. Lévy flight provides only a small and unstable contribution in the final scheduling framework. Thus, LSCBO’s cloud-scheduling advantage is best interpreted as the combined effect of a CBO search framework and a strong scheduling-specific reassignment operator, with stochastic perturbation serving as an auxiliary component.

The ablation results also caution against treating every added component as independently beneficial. Some reduced variants remain numerically close to the complete LSCBO setting, indicating that CBO phases, stochastic perturbation, and local search interact rather than contributing additively. This is why the paper assigns the core contribution to task-reassignment local search and treats Lévy perturbation as a secondary search-diversity mechanism.

## 5.2 Robustness and Generalizability

LSCBO’s performance advantage within the meta-heuristic scheduler comparison is robust across a wide range of experimental conditions. The weight sensitivity analysis confirms that LSCBO obtains the best objective rank under all five scalarization configurations. The VM heterogeneity analysis demonstrates that LSCBO has the lowest mean objective across MIPS ratios from 1.5:1 to 20:1. These findings suggest that LSCBO’s contribution is not an artifact of a single weight setting or VM configuration, while leaving broader cross-paradigm comparisons as a separate empirical question.

## 5.3 Limitations

Several limitations should be considered. First, the reported multi-cost metrics are computed from an analytic task/VM model; although all 12,780 formal rows were executed and confirmed to complete under a full CloudSim Plus simulation, the simulator uses a TimeShared scheduling policy and a linear power model, so the absolute cost values remain model-dependent and could shift under alternative simulator configurations or real datacenter telemetry. Second, the current power model is linear, which induces a structural dependence between makespan and energy; nonlinear power

models would be needed to fully decouple these objectives. Relatedly, because all schedulers execute the same task set on the same VM-price model, the execution-cost component varies only marginally across methods (Table 8) and contributes little discriminative signal to the scalarized objective; the observed advantage is therefore driven primarily by makespan, energy, and load balance rather than by cost. Third, following the standard format of cloud meta-heuristic scheduling studies, the main comparison focuses on population-based meta-heuristic schedulers rather than exact optimization, dispatching rules, or DRL-based schedulers; these provide useful boundary baselines but are outside the main comparative scope of this paper. Therefore, the results should be read as evidence of better performance within the evaluated cloud meta-heuristic scheduler family, not as a universal state-of-the-art claim across all scheduling paradigms. Fourth, the offline batch scheduling assumption limits direct applicability to dynamic online cloud environments because the present task-reassignment operator performs pre-execution assignment rather than runtime task migration. Fifth, the task-reassignment local search has been validated primarily within the CBO framework; its transferability to other meta-heuristics (GTO, PSO, GWO) remains an open empirical question.

## 6 Conclusion

This paper proposed LSCBO, a CBO-based scheduler that combines task-reassignment local search with Lévy-assisted stochastic exploration for multi-cost cloud task scheduling. The algorithm was comprehensively evaluated through two experimental series: (1) 29 CEC2017 global optimization functions to validate its fundamental optimization capability, and (2) formal multi-cost scheduling experiments across 5 task scales, 5 weight configurations, 3 VM heterogeneity regimes, component-level ablation, and perturbation mechanism comparisons.

The experimental results demonstrate that:

- LSCBO obtains the lowest average objective rank of 1.087 among the nine compared population-based meta-heuristic schedulers ( $p < 10^{-70}$ ), with per-scale objective improvements of 7.4%–22.6% and makespan improvements of 9.6%–30.1% over the standardized CBO baseline.
- On CEC2017 benchmarks, LSCBO obtains the lowest Friedman average rank of 1.86 among seven compared algorithms, with statistically significant wins over CBO, PSO, SA, and GWO ( $p < 0.001$ ) and statistical parity with DE ( $p = 0.059$ ) and GTO ( $p = 0.481$ ).
- The task-reassignment local search is the dominant performance driver, while stochastic perturbation provides a smaller and less stable auxiliary benefit.
- LSCBO’s ranking advantage within the evaluated meta-heuristic scheduler family is robust across all tested weight configurations and VM heterogeneity regimes.
- The evolution-path methodology provides an evidence-based approach for component-level ablation in meta-heuristic research.



Future work should: (1) implement Pareto-front methods to explore multi-objective trade-offs beyond single-point scalarization, (2) test the transferability of task-reassignment local search to other meta-heuristics, (3) investigate nonlinear power models to decouple makespan and energy objectives, (4) extend cost-decomposition curves across additional task scales and dynamic workload settings, (5) add a separate boundary-baseline study against dispatching rules, DRL-based schedulers, and exact or near-exact small-instance solvers, and (6) evaluate on real-world cloud workload traces with diverse distributional characteristics.

## Statements and Declarations

**Funding** No external funding was received for this work.

**Competing Interests** The authors declare no competing interests.

**Data Availability** The CSV files used to generate the current manuscript tables and figures, including the formal 12,780-row multi-cost comparison (with every row validated through a full CloudSim Plus simulation), evolution-path analysis, component-ablation analysis, CEC2017 ranking, weight-sensitivity data, VM heterogeneity data, and perturbation diagnostics, are organized with recomputation scripts and are deposited in the public project repository (<https://github.com/Lazywords2006/LSCBO>); they are additionally available from the corresponding author upon reasonable request during review.

**Code Availability** The LSCBO optimizer implementation, CloudSim Plus experiment entry points, result-aggregation scripts, and figure-generation scripts used in this study are publicly available at <https://github.com/Lazywords2006/LSCBO> to support reproducibility.

**Ethics Approval** Not applicable.

**Consent to Participate** Not applicable.

**Consent for Publication** Not applicable.

## References

- [1] Braun TD, Siegel HJ, Beck N, Bölöni LL, Maheswaran M, Reuther AI, Robertson JP, Theys MD, Yao B, Hensgen D, Freund RF. A comparison of eleven static heuristics for mapping a class of independent tasks onto heterogeneous distributed computing systems. *J Parallel Distrib Comput*, 61(6):810–837 (2001). <https://doi.org/10.1006/jpdc.2000.1714>
- [2] Kiliaan H, Mamo C, Paquet P. A coyote, *Canis latrans*, and badger, *Taxidea taxus*, interaction near Cypress Hills Provincial Park, Alberta. *Can Field-Nat*, 105(1):122–123 (1991). <https://doi.org/10.5962/p.357965>
- [3] Etminani K, Naghibzadeh M. A Min-Min Max-Min selective algorithm for grid task scheduling. *Proc 3rd IEEE/IFIP Int Conf Cent Asia Internet*, 1–7 (2007). <https://doi.org/10.1109/CANET.2007.4401694>

- [4] Kennedy J, Eberhart R. Particle swarm optimization. *Proc ICNN'95-Int Conf Neural Networks*, 4:1942–1948 (1995). <https://doi.org/10.1109/ICNN.1995.488968>
- [5] Goldberg DE. Genetic algorithms in search, optimization, and machine learning. *Addison-Wesley* (1989).
- [6] Storn R, Price K. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *J Glob Optim*, 11(4):341–359 (1997). <https://doi.org/10.1023/A:1008202821328>
- [7] Tasgetiren MF, Liang YC, Şevkli M, Gencyilmaz G. A particle swarm optimization algorithm for makespan and total flowtime minimization in the permutation flowshop sequencing problem. *Eur J Oper Res*, 177(3):1930–1947 (2007). <https://doi.org/10.1016/j.ejor.2005.12.024>
- [8] Mantegna RN. Fast, accurate algorithm for numerical simulation of Lévy stable stochastic processes. *Phys Rev E*, 49(5):4677–4683 (1994). <https://doi.org/10.1103/PhysRevE.49.4677>
- [9] Yang XS, Deb S. Cuckoo search via Lévy flights. *Proc 2009 World Congr Nat Biol Inspired Comput (NaBIC)*, 210–214 (2009). <https://doi.org/10.1109/NABIC.2009.5393690>
- [10] Heidari AA, Mirjalili S, Faris H, Aljarah I, Mafarja M, Chen H. Harris hawks optimization: algorithm and applications. *Future Gener Comput Syst*, 97:849–872 (2019). <https://doi.org/10.1016/j.future.2019.02.028>
- [11] Mirjalili S, Lewis A. The whale optimization algorithm. *Adv Eng Softw*, 95:51–67 (2016). <https://doi.org/10.1016/j.advengsoft.2016.01.008>
- [12] Xue J, Shen B. Dung beetle optimizer: a new meta-heuristic algorithm for global optimization. *J Supercomput*, 79:7305–7336 (2023). <https://doi.org/10.1007/s11227-022-04959-6>
- [13] Deng L, Liu S. Snow ablation optimizer: a novel metaheuristic technique for numerical optimization and engineering design. *Expert Syst Appl*, 225:120069 (2023). <https://doi.org/10.1016/j.eswa.2023.120069>
- [14] Abdel-Basset M, Mohamed R, Azeem SAA, Jameel M, Abouhawwash M. Kepler optimization algorithm: a new metaheuristic algorithm inspired by Kepler’s laws of planetary motion. *Knowl-Based Syst*, 268:110454 (2023). <https://doi.org/10.1016/j.knosys.2023.110454>
- [15] Qian Z, Zhang Y, Pu D, Xie G, Pu D, Ye M. A new hybrid improved Kepler optimization algorithm based on multi-strategy fusion and its applications. *Mathematics*, 13(3):405 (2025). <https://doi.org/10.3390/math13030405>

- [16] Abualigah L, Diabat A, Mirjalili S, Elaziz MA, Gandomi AH. The arithmetic optimization algorithm. *Comput Methods Appl Mech Eng*, 376:113609 (2021). <https://doi.org/10.1016/j.cma.2020.113609>
- [17] Hashim FA, Hussien AG. Snake optimizer: a novel meta-heuristic optimization algorithm. *Knowl-Based Syst*, 242:108320 (2022). <https://doi.org/10.1016/j.knosys.2022.108320>
- [18] Trojovský P, Dehghani M. Pelican optimization algorithm: a novel nature-inspired algorithm for engineering applications. *Sensors*, 22(3):855 (2022). <https://doi.org/10.3390/s22030855>
- [19] Faramarzi A, Heidarinejad M, Mirjalili S, Gandomi AH. Marine Predators Algorithm: a nature-inspired metaheuristic. *Expert Syst Appl*, 152:113377 (2020). <https://doi.org/10.1016/j.eswa.2020.113377>
- [20] Abualigah L, Elaziz MA, Sumari P, Geem ZW, Gandomi AH. Reptile Search Algorithm (RSA): a nature-inspired meta-heuristic optimizer. *Expert Syst Appl*, 191:116158 (2022). <https://doi.org/10.1016/j.eswa.2021.116158>
- [21] Wang L, Cao Q, Zhang Z, Mirjalili S, Zhao W. Artificial rabbits optimization: a new bio-inspired meta-heuristic algorithm for solving engineering optimization problems. *Eng Appl Artif Intell*, 114:105082 (2022). <https://doi.org/10.1016/j.engappai.2022.105082>
- [22] Dehghani M, Trojovský P. Osprey optimization algorithm: a new bio-inspired metaheuristic algorithm for solving engineering optimization problems. *Front Mech Eng*, 8:1126450 (2023). <https://doi.org/10.3389/fmech.2022.1126450>
- [23] Chintale P, Malviya RK, Merla NB, Chinna PPG, Desaboyina G, Sure TAR. Levy flight osprey optimization algorithm for task scheduling in cloud computing. *Proc 2024 Int Conf Intell Algorithms Comput Intell Syst (IACIS)*, 1–5 (2024). <https://doi.org/10.1109/IACIS61494.2024.10721633>
- [24] Khatab M, El-Gamel M, Saleh AI, El-Shenawy A, Rabie AH. Coyote and Badger Optimization (CBO): a natural inspired meta-heuristic algorithm based on cooperative hunting. *Commun Nonlinear Sci Numer Simul*, 140:108333 (2025). <https://doi.org/10.1016/j.cnsns.2024.108333>
- [25] Xie R, Li S, Wu F, Wu L, Xiong X. An enhanced dung beetle optimizer for cloud task schedule. *Proc 4th Int Conf Appl Math Model Intell Comput (CAMMIC 2024)*, 13219:80 (2024). <https://doi.org/10.1117/12.3036506>
- [26] Ghafari R, Mansouri N. E-AVOA-TS: enhanced African vultures optimization algorithm-based task scheduling strategy for fog–cloud computing. *Sustain Comput Inform Syst*, 40:100918 (2023). <https://doi.org/10.1016/j.suscom.2023.100918>

- [27] Rengaraj alias Muralidharan R, Latha K. Gorilla troops optimizer based fault tolerant aware scheduling scheme for cloud environment. *Intell Autom Soft Comput*, 35(2):1923–1937 (2023). <https://doi.org/10.32604/iasc.2023.029495>
- [28] Ferahtia S, Houari A, Rezk H, Djerioui A, Machmoum M, Motahhir S, Ait-Ahmed M. Red-tailed hawk algorithm for numerical optimization and real-world problems. *Sci Rep*, 13:12950 (2023). <https://doi.org/10.1038/s41598-023-38778-3>
- [29] Qin X, Li S, Tong J, Xie C, Zhang X, Wu F, Xie Q, Ling Y, Lin G. ERTS scheduler: enhanced red-tailed hawk algorithm for multi-cost optimization in cloud task scheduling. *Artif Intell Rev*, 57:328 (2024). <https://doi.org/10.1007/s10462-024-10945-6>
- [30] Ebrahimi Mood S, Sourì A, Inanc N, Li KC. CEEO: an innovative Coulomb-enhanced equilibrium optimizer for task scheduling of cloud services in IoT environments. *Computing*, 107:138 (2025). <https://doi.org/10.1007/s00607-025-01495-y>
- [31] Gunduz A, Senan S, Gurkas-Aydin Z. A hybrid DECSO algorithm for efficient multi-objective task scheduling in cloud computing environments. *Cluster Comput*, 28:848 (2025). <https://doi.org/10.1007/s10586-025-05586-5>
- [32] Sabipour MR, Sayadnavard MH. A chaotic RIME optimization algorithm based on generalized quadratic interpolation with Lévy flight for task scheduling in cloud computing. *J Soft Comput Inf Technol*, 14(1):57–67 (2025). <https://doi.org/10.22034/jscit.2025.220413>
- [33] Zhu M, Li J, Yang X. A hybrid SAO and RIME optimizer for global optimization and cloud task scheduling. *Biomimetics*, 10(10):690 (2025). <https://doi.org/10.3390/biomimetics10100690>
- [34] Abdollahzadeh B, Soleimani Gharehchopogh F, Mirjalili S. Artificial gorilla troops optimizer: a new nature-inspired metaheuristic algorithm for global optimization problems. *Int J Intell Syst*, 36:5887–5958 (2021). <https://doi.org/10.1002/int.22535>
- [35] Mirjalili S, Mirjalili SM, Lewis A. Grey wolf optimizer. *Adv Eng Softw*, 69:46–61 (2014). <https://doi.org/10.1016/j.advengsoft.2013.12.007>
- [36] Moscato P. On evolution, search, optimization, genetic algorithms and martial arts: towards memetic algorithms. *Caltech Concurrent Computation Program, C3P Report 826* (1989).
- [37] Mao H, Schwarzkopf M, Bojja Venkatakrishnan S, Meng Z, Alizadeh M. Learning scheduling algorithms for data processing clusters. *Proc ACM SIGCOMM 2019*, 270–288 (2019). <https://doi.org/10.1145/3341302.3342080>

- [38] Mao H, Alizadeh M, Menache I, Kandula S. Resource management with deep reinforcement learning. *Proc 15th ACM Workshop Hot Topics Netw (HotNets)*, 50–56 (2016). <https://doi.org/10.1145/3005745.3005750>
- [39] Deb K, Pratap A, Agarwal S, Meyarivan T. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans Evol Comput*, 6(2):182–197 (2002). <https://doi.org/10.1109/4235.996017>
- [40] Zhang Q, Li H. MOEA/D: a multiobjective evolutionary algorithm based on decomposition. *IEEE Trans Evol Comput*, 11(6):712–731 (2007). <https://doi.org/10.1109/TEVC.2007.892759>
- [41] Calheiros RN, Ranjan R, Beloglazov A, De Rose CAF, Buyya R. CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. *Softw Pract Exp*, 41(1):23–50 (2011). <https://doi.org/10.1002/spe.995>
- [42] Silva Filho MC, Oliveira RL, Monteiro CC, Inacio PRM, Freire MM. CloudSim Plus: a cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness. *Proc 2017 IFIP/IEEE Symp Integr Netw Serv Manag (IM)*, 400–406 (2017). <https://doi.org/10.23919/INM.2017.7987304>
- [43] Tanabe R, Fukunaga AS. Improving the search performance of SHADE using linear population size reduction. *Proc 2014 IEEE Congr Evol Comput (CEC)*, 1658–1665 (2014). <https://doi.org/10.1109/CEC.2014.6900380>
- [44] Hansen N, Ostermeier A. Completely derandomized self-adaptation in evolution strategies. *Evol Comput*, 9(2):159–195 (2001). <https://doi.org/10.1162/106365601750190398>
- [45] Demšar J. Statistical comparisons of classifiers over multiple data sets. *J Mach Learn Res*, 7:1–30 (2006).
- [46] Friedman M. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *J Am Stat Assoc*, 32(200):675–701 (1937). <https://doi.org/10.1080/01621459.1937.10503522>
- [47] Wilcoxon F. Individual comparisons by ranking methods. *Biometrics Bull*, 1(6):80–83 (1945). <https://doi.org/10.2307/3001968>
- [48] Iman RL, Davenport JM. Approximations of the critical region of the Friedman statistic. *Commun Stat-Theory Methods*, 9(6):571–595 (1980). <https://doi.org/10.1080/03610928008827904>
- [49] Kirkpatrick S, Gelatt Jr CD, Vecchi MP. Optimization by simulated annealing. *Science*, 220(4598):671–680 (1983). <https://doi.org/10.1126/science.220.4598.671>

- [50] Awad NH, Ali MZ, Liang JJ, Qu BY, Suganthan PN. Problem definitions and evaluation criteria for the CEC 2017 special session and competition on single objective bound constrained real-parameter numerical optimization. *Nanyang Technological University*, Tech. Rep. (2016).